

DSA & Advanced DSA Roadmap – Proposed Structure

Vision

The objective of this roadmap is to transform students from basic programmers into placement-ready problem solvers through a structured progression of:

Implementation & Internals → Pattern Mastery → Problem Solving → Advanced Optimization

Unlike traditional DSA courses that focus only on solving coding questions, this roadmap ensures students first understand how a data structure works internally, implement it from scratch, and then master the patterns used in coding interviews and assessments.

Roadmap Overview

The curriculum is divided into 17 phases grouped under three major tracks.

Track 1: Linear Data Structures

Phase 0 – Problem Solving Foundation

Topics

- What is an Algorithm?
- Time Complexity
- Space Complexity
- Big O Notation
- Complexity Analysis
- Input & Output Optimization
- Dry Run Techniques
- Recursion Fundamentals

Goal

Build analytical thinking and complexity awareness before entering DSA.

Phase 1 – Arrays

Implementation & Internals

- Static Arrays
- Dynamic Arrays
- Memory Layout
- Resizing Strategy
- ArrayList Internals
- Amortized Analysis

Pattern Mastery

- Traversal



- Frequency Counting
- Prefix Sum
- Difference Array
- Two Pointers
- Sliding Window
- Kadane's Algorithm
- Merge Intervals
- Cyclic Sort

Phase 2 – Strings

Implementation & Internals

- ASCII
- Unicode
- UTF-8
- String Storage
- String Immutability
- StringBuilder Internals

Pattern Mastery

- Character Frequency
- Palindrome
- Anagram
- Two Pointers
- Sliding Window
- String Hashing Basics

Phase 3 – Linked Lists

Implementation & Internals

- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Node Design
- Pointer Manipulation

Pattern Mastery

- Fast & Slow Pointer
- Reverse Linked List
- Merge Lists
- Cycle Detection

- Dummy Node Pattern

Phase 4 – Stack

Implementation & Internals

- Array Stack
- Linked List Stack
- Function Call Stack
- Recursion Stack

Pattern Mastery

- Balanced Parentheses
- Monotonic Stack
- Next Greater Element
- Histogram Problems
- Expression Evaluation

Phase 5 – Queue

Implementation & Internals

- Queue
- Circular Queue
- Deque
- Priority Queue Basics

Pattern Mastery

- BFS Foundation
- Sliding Window Maximum
- Monotonic Queue
- Multi-Source BFS

Phase 6 – Hashing

Implementation & Internals

- Hash Functions
- Buckets
- Collision Handling
- Rehashing
- Load Factor

Pattern Mastery

- Frequency Counting
- Pair Sum



- Prefix Hashing
- Hash + Sliding Window
- Set-Based Problems

Track 2: Core Algorithms

Phase 7 – Searching Algorithms

Implementation & Internals

- Linear Search
- Binary Search
- Search Space Reduction

Pattern Mastery

- Lower Bound
- Upper Bound
- Binary Search on Answer
- Rotated Arrays

Phase 8 – Sorting Algorithms

Implementation & Internals

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort
- Counting Sort

Pattern Mastery

- Custom Sorting
- Sorting + Greedy
- Interval Sorting

Phase 9 – Recursion

Implementation & Internals

- Call Stack
- Recursion Tree
- Tail Recursion
- State Space Exploration

Pattern Mastery



- Recursive Thinking
- Divide & Conquer
- Tree Recursion

Phase 10 – Backtracking

Implementation & Internals

- Decision Trees
- State Space Search

Pattern Mastery

- Subsets
- Permutations
- Combinations
- N Queens
- Sudoku Solver

Track 3: Non-Linear Data Structures

Phase 11 – Trees

Implementation & Internals

- Binary Trees
- BST
- Recursive Traversals
- Iterative Traversals

Pattern Mastery

- DFS
- BFS
- Tree Height
- Diameter
- Path Sum
- Lowest Common Ancestor
- Tree Construction

Phase 12 – Heap

Implementation & Internals

- Min Heap
- Max Heap
- Heapify
- Priority Queue

Pattern Mastery

- Top K Elements
- K Closest Elements
- Running Median
- Merge K Sorted Lists

Phase 13 – Trie**Implementation & Internals**

- Trie Node Design
- Insert
- Search
- Prefix Search

Pattern Mastery

- Auto Complete
- Dictionary Search
- Prefix Matching

Phase 14 – Graphs**Implementation & Internals**

- Adjacency Matrix
- Adjacency List
- Directed Graphs
- Undirected Graphs
- Weighted Graphs

Pattern Mastery

- BFS
- DFS
- Connected Components
- Cycle Detection
- Bipartite Graphs
- Topological Sort
- Union Find (DSU)
- Shortest Path

Advanced Algorithms (ADSA)**Phase 15 – Greedy Algorithms****Implementation & Internals**

- Greedy Choice Property
- Local vs Global Optimum

Pattern Mastery

- Interval Scheduling
- Meeting Rooms
- Fractional Knapsack
- Huffman Coding

Phase 16 – Dynamic Programming**Implementation & Internals**

- Memoization
- Tabulation
- State Design
- Transition Design

Pattern Mastery

- 1D DP
- 2D DP
- Knapsack Pattern
- LIS Pattern
- Grid DP
- Partition DP
- Tree DP
- Bitmask DP (Advanced)

Phase 17 – Competitive Programming & Advanced Optimization**Range Query Techniques**

- Prefix Sum
- Difference Array
- Range Updates

Advanced Searching

- Binary Search on Answer
- Parametric Search

Mathematical Techniques

- Modular Arithmetic
- Fast Exponentiation
- GCD
- Sieve

Advanced Tree Techniques

- Euler Tour
- Ancestor Queries
- Subtree Queries

Advanced Data Structures

- Union Find (DSU)
- Segment Tree
- Lazy Propagation

Goal

Prepare students for:

- Premium placement rounds
- HackWithInfy
- Product company coding tests
- Competitive programming contests

Learning Methodology

Each phase follows the same structure:

1. Learn Internals
2. Implement from Scratch
3. Master Patterns
4. Solve Practice Problems
5. Complete Curated LeetCode Sheet
6. Take Phase Assessment

Expected Outcome

Upon completion, students will:

- Understand data structures beyond API usage.
- Implement core structures from scratch.
- Recognize and apply interview patterns.
- Solve Easy, Medium, and selected Hard LeetCode problems.
- Perform well in service-company placements.
- Compete confidently in product-company coding assessments.
- Build a strong foundation for advanced software engineering and competitive programming.